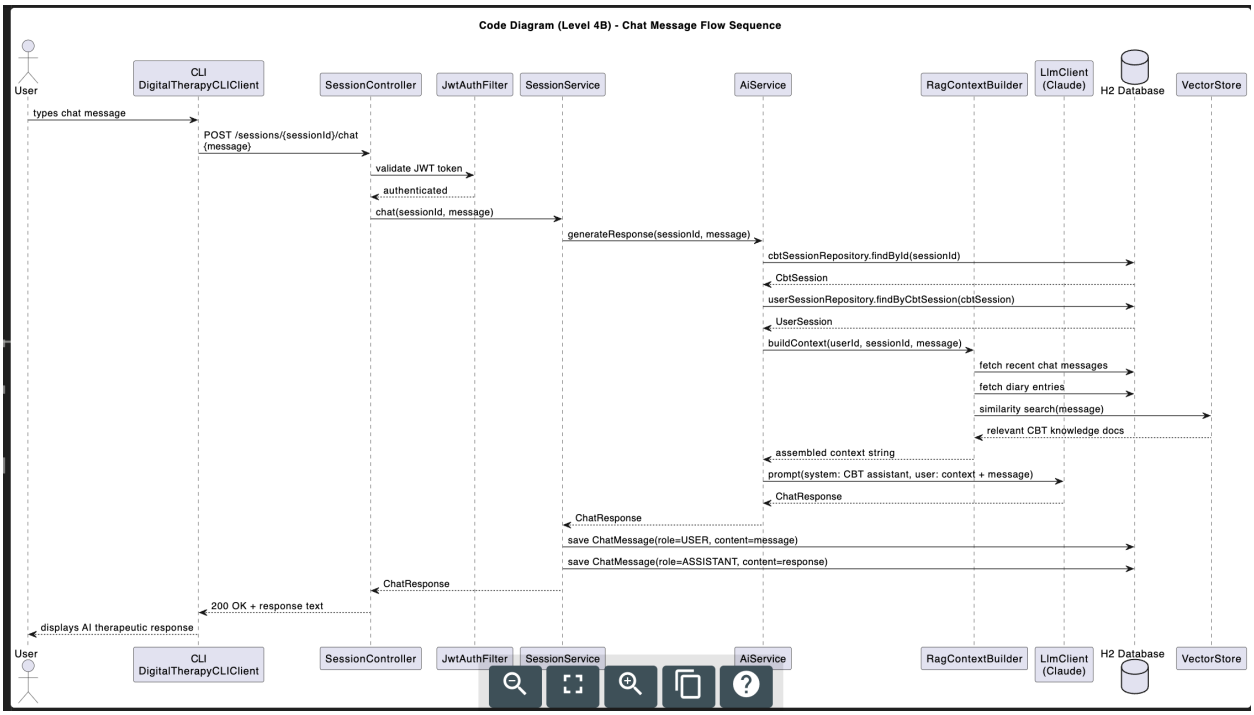
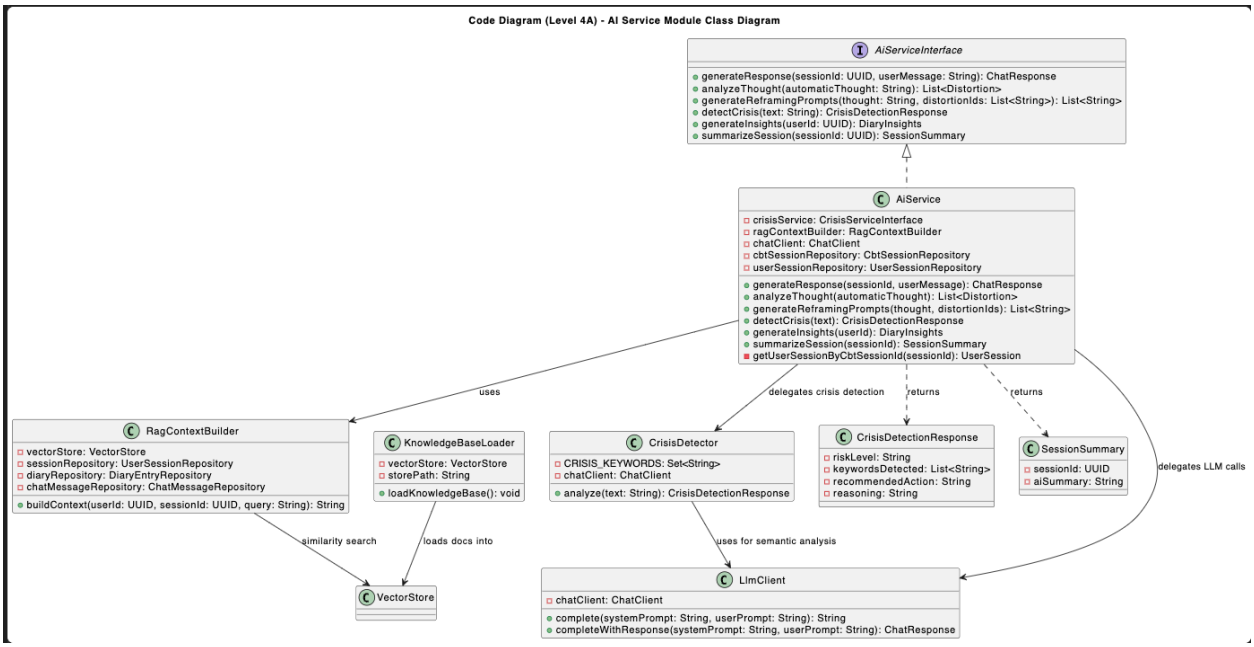
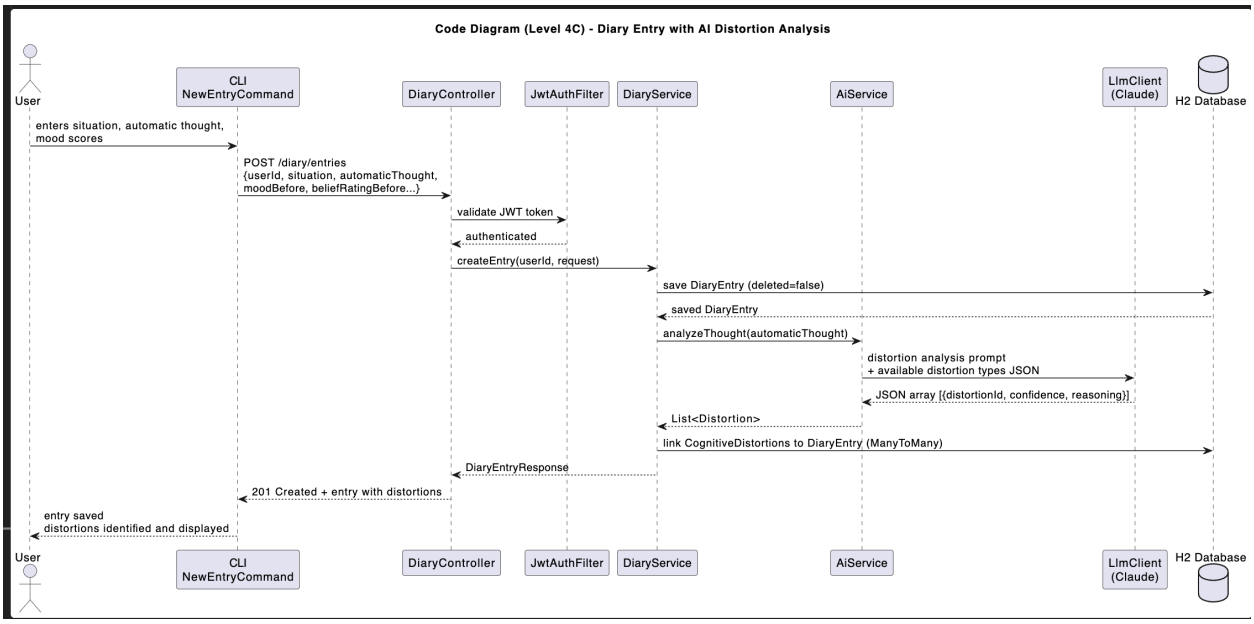

Code Diagram (Level 4B) - Chat Message Flow Sequence
User
CLI DigitalTherapyCLIClient
SessionController
JwtAuthFilter
SessionService
AiService
RagContextBuilder
LlmClient (Claude)
H2 Database
VectorStore
types chat message
POST /sessions/{sessionId}/chat (message)
validate JWT token
authenticated
chat(sessionId, message)
generateResponse(sessionId, message)
cbtSessionRepository.findById(sessionId)
CbtSession
userSessionRepository.findByCbtSession(cbtSession)
UserSession
buildContext(userId, sessionId, message)
fetch recent chat messages
fetch diary entries
similarity search(message)
relevant CBT knowledge docs
assembled context string
prompt(system: CBT assistant, user: context + message)
ChatResponse
ChatResponse
save ChatMessage(role=USER, content=message)
save ChatMessage(role=ASSISTANT, content=response)
ChatResponse
200 OK + response text
displays AI therapeutic response


Code Diagram (Level 4A) - AI Service Module Class Diagram
AiServiceInterface
generateResponse(sessionId: UUID, userMessage: String): ChatResponse
analyzeThought(automaticThought: String): List<Distortion>
generateReframingPrompts(thought: String, distortionIds: List<String>): List<String>
detectCrisis(text: String): CrisisDetectionResponse
generateInsights(userId: UUID): DiaryInsights
summarizeSession(sessionId: UUID): SessionSummary
AiService
crisisService: CrisisServiceInterface
ragContextBuilder: RagContextBuilder
chatClient: ChatClient
cbtSessionRepository: CbtSessionRepository
userSessionRepository: UserSessionRepository
generateResponse(sessionId, userMessage): ChatResponse
analyzeThought(automaticThought): List<Distortion>
generateReframingPrompts(thought, distortionIds): List<String>
detectCrisis(text): CrisisDetectionResponse
generateInsights(userId): DiaryInsights
summarizeSession(sessionId): SessionSummary
getUserSessionByCbtSessionId(sessionId): UserSession
uses
delegates crisis detection
returns
returns
delegates LLM calls
RagContextBuilder
vectorStore: VectorStore
sessionRepository: UserSessionRepository
diaryRepository: DiaryEntryRepository
chatMessageRepository: ChatMessageRepository
buildContext(userId: UUID, sessionId: UUID, query: String): String
KnowledgeBaseLoader
vectorStore: VectorStore
storePath: String
loadKnowledgeBase(): void
CrisisDetector
CRISIS_KEYWORDS: Set<String>
chatClient: ChatClient
analyze(text: String): CrisisDetectionResponse
CrisisDetectionResponse
riskLevel: String
keywordsDetected: List<String>
recommendedAction: String
reasoning: String
SessionSummary
sessionId: UUID
aiSummary: String
similarity search
loads docs into
uses for semantic analysis
VectorStore
LlmClient
chatClient: ChatClient
complete(systemPrompt: String, userPrompt: String): String
completeWithResponse(systemPrompt: String, userPrompt: String): ChatResponse


Code Diagram (Level 4C) - Diary Entry with AI Distortion Analysis
User
CLI
NewEntryCommand
DiaryController
JwtAuthFilter
DiaryService
AiService
LlmClient
(Claude)
H2 Database
enters situation, automatic thought,
mood scores
POST /diary/entries
{userId, situation, automaticThought,
moodBefore, beliefRatingBefore...}
validate JWT token
authenticated
createEntry(userId, request)
save DiaryEntry (deleted=false)
saved DiaryEntry
analyzeThought(automaticThought)
distortion analysis prompt
+ available distortion types JSON
JSON array [{distortionId, confidence, reasoning}]
List<Distortion>
link CognitiveDistortions to DiaryEntry (ManyToMany)
DiaryEntryResponse
201 Created + entry with distortions
entry saved
distortions identified and displayed
User
CLI
NewEntryCommand
DiaryController
JwtAuthFilter
DiaryService
AiService
LlmClient
(Claude)
H2 Database